Working with Claude: efficient, safe, compliant and predictable

This repository collects practical guidance for using Claude (claude.ai, Cowork, Claude Code and the Claude API) in real projects. The aim is to get the most value out of Claude while keeping cost low, results predictable, data safe and the work compliant with regulation such as the EU AI Act.

Each document covers one area that a team needs to take a position on before AI use scales up. Together they work as a checklist for setting up a new project and as a reference when something needs to change.

Why this matters

A language model is not a normal software component. It is probabilistic, it is billed per token, it can act on your files and systems through tools, and depending on what it is used for it may fall under legal requirements. None of this is a problem if it is decided on deliberately. Most problems come from defaults that nobody chose: the most expensive model for every task, rules that only exist in a chat, no record of what the agent did, or a use case that turns out to be high-risk after launch.

The areas

1. Right model for the task (cost and token usage)

What: Choose the cheapest model that can do the job and escalate only when it is not enough. Small models for mechanical, easily checked work; mid-size models for everyday coding and writing; large models where a mistake is expensive, such as architecture, security, hard debugging and review before merge. Use caching, batch processing and cheaper sub-agents where possible.

Why: Output prices differ by roughly a factor of 100 between the smallest and largest models. Most tokens in a typical project go to reading, searching and summarising, which needs little reasoning. Matching the model to the task can cut cost substantially without weakening support for the hard problems. Too small a model can also cost more if it needs several attempts, so the choice should be measured, not guessed.

2. Determinism (predictable results)

What: Claude does not give identical answers to identical input, and this cannot be switched off. Predictability comes from what is built around it: let ordinary code handle anything that must be exact, keep decisions and rules in files Claude always reads (for example `CLAUDE.md`), require fixed output formats (structured output / JSON schema), check results automatically with tests and hooks, pin model versions, and store answers that must be reproducible.

Why: Calculations, permissions, assessments and anything involving personal data must behave the same way every time and be explainable afterwards. A useful rule of thumb: if it would worry you that the answer could be slightly different next time, it should not be decided by the model in the moment.

3. The agentic harness (safety and control)

What: The harness is everything around the model that turns it into an agent: the loop that runs tools, the context it reads (`CLAUDE.md`, skills, memory, handoff files) and the guardrails that decide what actually happens (permissions with allow/ask/deny, the sandbox, hooks). This area covers how to configure these, where each piece lives (personal machine, repo, GitHub, Claude project) and how to layer protection.

Why: The model proposes; the harness decides and executes. Instructions can be missed, but a hook or a deny rule runs every time. Irreversible actions such as pushing, merging or commenting publicly should require approval or be blocked. Important rules should be enforced in several layers (agent hook, git hook,

repository ruleset) so that one gap does not become an incident. Moving state out of the chat into issues, commits and handoff files also makes sessions resumable and reviewable.

4. Observability (knowing what Claude did)

What: Claude Code and Cowork have built-in OpenTelemetry support for metrics, events and (in beta) traces. This area covers how to turn it on, which backend to use, and which content is sent by default versus only when explicitly enabled.

Why: Without data you cannot tell whether model choices actually save money, which tools fail, where time goes in a slow session, or how often permission requests are denied. By default only numbers and metadata are sent, not the text of prompts or responses. Keeping it that way is the safe choice when prompts or tool output could contain information that does not belong in a log.

5. Regulatory compliance (EU AI Act)

What: The EU AI Act (Regulation (EU) 2024/1689, as amended by the Digital Omnibus on AI, (EU) 2026/1744) regulates AI by risk level based on what a system is used for, not what technology it uses. This area summarises the risk levels, the roles (provider, deployer, importer, distributor), the timeline and how to build the requirements into a project from day one.

Why: Prohibitions, AI literacy and transparency requirements already apply. High-risk requirements (for example in recruitment, credit scoring and education, including assessment of students) apply from December 2027 or August 2028. Classifying a use case early is cheap; discovering a high-risk system after launch is expensive. Role changes are a common trap: rebranding, substantially modifying or repurposing a system can make you the provider with full responsibility. Design choices, such as keeping a human in charge of decisions, can keep a system out of the high-risk category.

6. Data protection and security

What: What data may be sent to Claude, what may be logged, how secrets and personal data are kept out of prompts, repos and telemetry, and how this aligns with GDPR and information security policies.

Why: The AI Act does not replace GDPR. Personal data in prompts, tool output or logs is still personal data. Clear rules, enforced by code (for example a check that blocks personal identity numbers before commit), protect both users and the organisation.

7. Adapting Claude to the team

What: Shared instructions, skills, project files, plugins and connectors that make Claude follow the team's conventions, templates and decisions.

Why: Adaptation is what makes Claude both more useful and more consistent. The same task done the same way each time means less correction, fewer tokens spent re-explaining context, and output that matches the organisation's standards.

How to use this repository

- **Starting a project:** go through the areas in order and record a decision for each, even when the answer is "minimal risk" or "default settings are fine".
- **During the project:** use the documents as reference when adding tools, changing models or expanding what the AI is used for.
- **When something changes:** re-check the compliance and data protection areas whenever the purpose of a system changes. A new purpose can mean a new risk level.

Documents

Area	Document
Model choice and cost	`model-selection.md`
Determinism	`deterministic-work.md`
Agentic harness	`agentic-harness.md`
Observability	`observability.md`
EU AI Act	`ai-act.md`
Data protection and security	`data-protection.md`
Adapting Claude	`customisation.md`

Disclaimer

Prices, product features and regulatory dates change. Each document states when its sources were retrieved; verify against the sources before making decisions. The AI Act summary is guidance, not legal advice.